A Performance and Area Exploration of Optical Memories in the Computing Stack

Parth Deshpande, Aidan Mulvaney

CS/ECE 752 - Advanced Computer Architecture I

Professor Matthew Sinclair

March 25, 2026

Abstract

For decades, the gap between compute capability and memory bandwidth has widened exponentially. Historically, the industry addressed these issues by making advancements in processing to hide the memory latency, to limited success. Recent research shows that leveraging photons in waveguides and memory storage devices versus electrons in traditional technologies has the potential to, at scale, operate at frequencies upward of 10 GHz, outpacing the performance of any single processor to date.

This paper explores the broader potential of optical memories by analyzing the performance of optical caches and optical main memories across various processing architectures. Our findings indicate that while optical main memories exhibit exceptional performance, their large physical size makes them impractical for modern computing systems. Conversely, optical caches strike an optimal balance between size and performance, outperforming existing memory technologies. We further investigate strategic placements for optical caches within the computing stack, highlighting their potential to reshape the memory hierarchy.

Contents

1 Background **4**

2 Methodology **7**

2.1 Traditional Hierarchy 9

2.2 Optical Main Memory 9

2.3 Optical Caches 10

2.4 Optical Caches in Multiprocessors 11

3 Tasks and Contributions **12**

3.1 Task 1: Collection of Physical Parameters 12

3.2 Task 2: Creation of Optical Memory Models in gem5 12

3.3 Task 3: Creation of Configuration Scripts 12

3.4 Task 4: Modifying Hopscotch for gem5 Compatibility 13

3.5 Task 5: Collecting and Interpreting Data 13

4 Results **13**

4.1 In Order Superscalar CPU Results 14

4.2 Out of Order Superscalar CPU Results 14

4.3 Area Considerations 15

4.4 Multiprocessing Results 16

5 Related Work **17**

6 Conclusion **18**

7 Future Work **19**

1 Background

The exponential increase in the energy-delay product associated with data movement and memory access highlights the growing memory bottleneck in von Neumann computational systems [1]. At the same time, Moore's Law, which predicts exponential growth in the number of transistors, is increasingly challenged [2, 3] due to escalating costs and technical complexities in fabricating transistors at extremely small nanometer scales, where quantum effects compromise reliability and make further scaling difficult [4]. These challenges limit the enhancement of computing performance, while the demand for more computational power continues to grow, driven by advancements in artificial intelligence, machine learning, and deep learning applications [5]. These factors exacerbate the von Neumann bottleneck, resulting in increased energy consumption and latency. Figure 1 from the Hot Chips 2023 symposium [6] shows the difference in growth rates of compute power and memory bandwidth. To address these limitations, researchers are exploring alternative computing paradigms such as quantum computing [7], non-volatile memory-based computing [8], neuromorphic computing [9], and optical computing [10] to overcome the constraints of traditional silicon-based technologies.

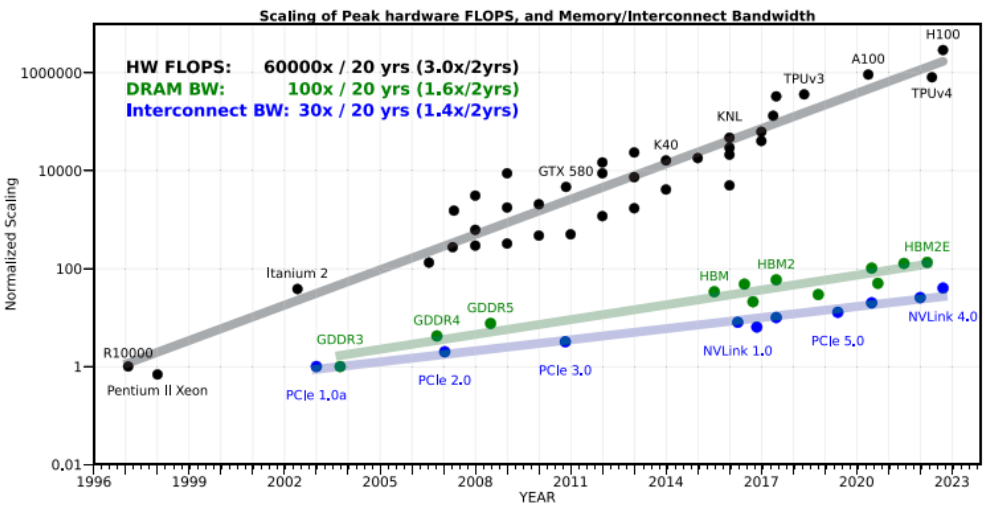

Figure 1: Rate of growth of compute power and memory bandwidth[6]

CMOS-based systems encounter critical challenges as interconnect scaling lags behind transistor scaling, creating speed and energy efficiency bottlenecks [11]. As technology advances, electronic memory faces further issues, such as increased bitline and wordline capacitance, and higher resistance in longer

interconnects, exacerbating these limitations [12]. In contrast, optical systems use waveguides for low-loss data transfer, effectively addressing the scaling constraints of electrical interconnects. Photons, traveling faster than electrons, allow for rapid data transmission, especially when distributing information to multiple destinations, resulting in high bandwidth and minimal interference [13, 14]. Additionally, since photons do not adhere to the Pauli exclusion principle, wavelength-division multiplexing (WDM) can be employed, enhancing computational efficiency, bandwidth, and throughput [15].

To address the challenges of storing light, particularly due to the neutral charge of photons, which prevents replication of capacitor-based electronic memory designs, researchers have developed several methods for light-based storage. These approaches primarily leverage bistability—either through engineered optical resonances (artificial cavities) [16] or the inherent bistable properties of specific materials [17]. For a system to achieve optical bistability and function as a memory device, two key conditions must be met. The system must exhibit two distinct, stable states to represent logical "1" and "0", and it must allow controlled switching between these states under specific conditions. There are several ways of doing this, as shown in Figure 2; including master-slave configuration, feedback loop scheme, injection locking technique, and phase-change materials, each offering unique methods for achieving bistability and enabling optical memory functionality [18].

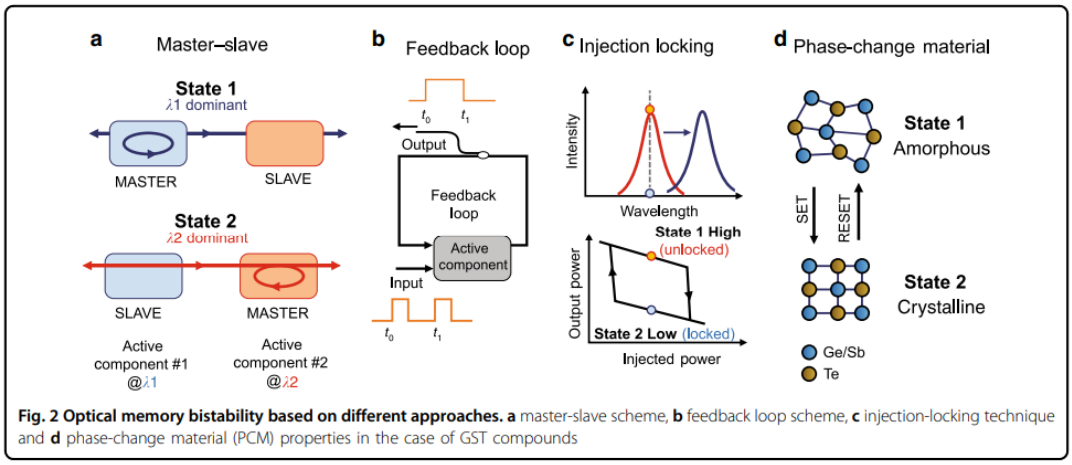

Figure 2: Different ways of storing light[18]

For the sake of this project, we will focus on one specific type of optical memory - Micro-Ring Resonators (MRRs). As shown in Figure 3, micro-ring resonators are compact, circular optical devices

that manipulate light in integrated photonic circuits, leveraging resonance effects to control and filter optical signals efficiently [19]. They consist of a circular waveguide (ring) that traps light, linear bus waveguides for coupling light in and out, and phase shifters to tune resonance dynamically. The MRR has input, through, and drop ports: light injected at the input port couples into the ring via evanescent coupling when the waveguide and ring are in close proximity. At resonance, the optical path length in the ring matches an integer multiple of the light wavelength, causing constructive interference. This results in strong coupling of resonant light to the drop port, while non-resonant wavelengths pass through the bus waveguide with minimal loss. MRRs can be dynamically tuned using electrical signals. Thus, there are two stable states and the ability to switch between them representing a photonic memory. Figure 4 shows an array of MRR bitcells that can be created resembling a CMOS RAM array. There are word lines and bit lines but they will be optical waveguides instead of electrical interconnects. This gives the system the ability leverage various properties of photons [20].

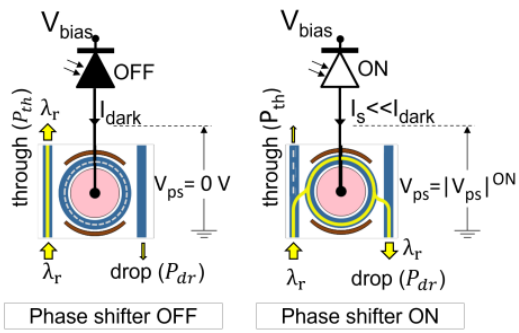

Figure 3: MRR

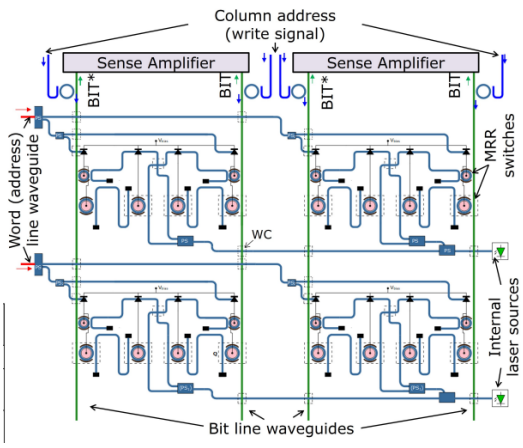

Figure 4: Array of Bitcells

The processing of data needs to happen in the electrical processor. Thus, when the processor requests data from the optical memory, it needs to be converted to electrical signals using photodiodes. Once light is converted to electrical signals, it is ready for processing through ALUs and other functional units. Table 1 shows the main advantage of using optical memories. The read latency of O-SRAM or an optical cache is less than 50 picoseconds which corresponds to greater than 20GHz of speed. This is in contrast with electrical DRAM and SRAM speeds which gives a maximum of 5Ghz speed. Even in terms of

energy efficiency, optical memories outperform electrical ones [21]. Although there is an area cost which will be discussed later, the advantage of optical memories is clear - faster speeds due to reduced read latency, higher bandwidth due to wave division multiplexing. Thus, we have an optical memory sytem setup which mimics the electrical memory system with row and column addressing and sense amplifiers with a much higher speed and bandwidth.

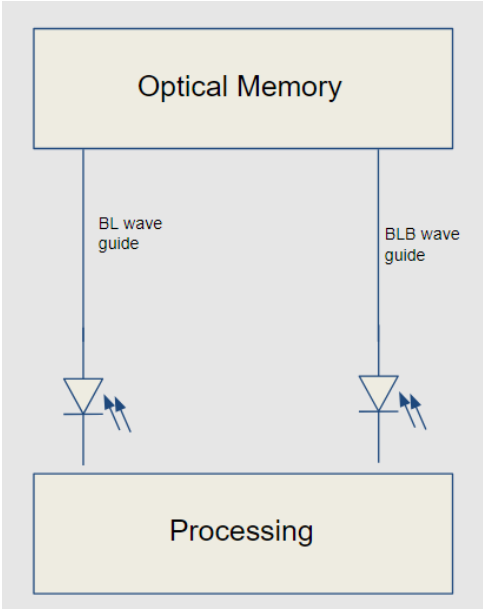

Figure 5: Converting light to electrical signals for processing

Memory Type	**Read Latency(ps)**	**Energy Efficiency (pJ/bit)**	**Area(μm^2)**
eDRAM (28nm HKMG)	2000	19	0.0035
SRAM (7nm FINFET)	300	5	0.01
SRAM Cache (64-byte block)	1660	0.35	0.055
O-SRAM	<50	0.0032	6.2
Optical Cache (8kB)	<50	0.143	120

Table 1: Comparison of various parameters

2 Methodology

To evaluate the efficacy of optical memory as a suitable replacement for electronic memories in the computing stack, we have proposed several different memory schemes. In Section 2.1, our traditional memory hierarchy will be presented as a baseline. In Section 2.2, we will explore replacing the entire

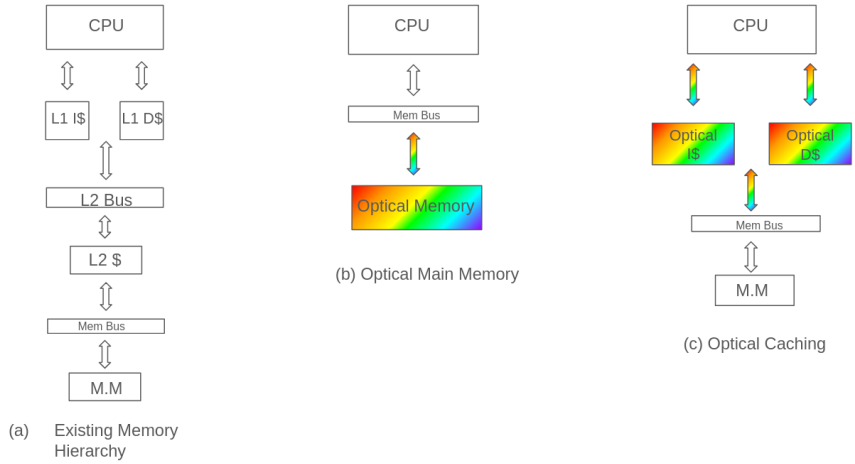

Figure 6: Collection of single core hierarchies

memory subsystem with a large optical main memory. In Section 2.3, we will replace the cache hierarchy with a pair of optical caches, but leave main memory intact. To explore the efficacy of optical memories in multiprocessor systems, in Section 2.4 we will replace the unified L2 cache between cores with an optical cache. For each of our proposed systems, we evaluated and modeled them in gem5 Syscall Emulation mode. For the single core systems, we operated the processor at 4 GHz, in both the X86MinorCPU and X86O3CPU models. We also evaluated our single core system at multiple processor widths, as we believed our optical memories could support systems with exceedingly high processor widths. With the multiprocessor systems, we evaluated the performance of both 2-core and 4-core X86O3CPU models, each running at 4 GHz, with a processor width of 8.

As the changes we make to the computing stack are in the memory subsystem, we desired to find a benchmark which adequately evaluates the performance of the memory subsystem. For this purpose, we made use of the Hopscotch benchmark suite [22]. We preferred this benchmark suite over other memory performance suites, as the Hopscotch suite employs many smaller benchmarks, which reduces the compute overhead, while amplifying the impact of memory operations. Hopscotch provides a wide range of access patterns – sequential, strided, or random – in read, write, and read/write benchmarks. The full list of benchmarks offered by the Hopscotch suite can be seen in Figure 6.

Kernel Name	Operation (i = outer loop index, j = inner loop index)	Read Pattern	Write Pattern
r_seq_ind	$x = a[i]$	sequential	–
r_seq_reduce	$x \mathrel{+}= a[i]$	sequential	–
r_rand_ind	$idx = rand_gen(i)$ $x = a[idx]$	random	–
r_rand_pchase	$p = *p$	random	–
r_stride_<k>	$x = a[i]; i \mathrel{+}= k;$	strided	–
r_tile	$x = a[i + j]; i \mathrel{+}= k;$	tunable	–
w_seq_memset	$memset(a, x, W)$	–	sequential
w_seq_fill	$a[i] = x$	–	sequential
w_rand_ind	$idx = rand_gen(i)$ $a[idx] = x$	–	random
w_stride_<k>	$a[i] = x; i \mathrel{+}= k;$	–	strided
w_tile	$a[i + j] = x; i \mathrel{+}= k;$	–	tunable
rw_seq_inc	$a[i]$++	sequential	sequential
rw_gather	$a[i] = b[idx[i]]$	sequential + random	sequential
rw_scatter	$a[idx[i]] = b[i]$	sequential	random
rw_scatter_gather	$a[idx_1[i]] = b[idx_2[i]]$	sequential + random	random
rw_tile	$a[i + j] = b[i + j];$ $i \mathrel{+}= k;$	tunable	tunable

Table 2: All Hopscotch benchmarks and their access patterns[22]

2.1 Traditional Hierarchy

To properly evaluate our novel memory changes, we need to first evaluate a traditional memory subsystem, so that we have a baseline for which we can compare the performance of optical memories. Our traditional hierarchy, which can be seen in Figure 6a, consists of two L1 caches, a 2 way set associative, 16 KB instruction cache, and a 2 way set associative, 64 KB data cache. The access latencies of the L1 caches are kept at a single cycle. We connect these L1 caches to a unified L2 cache, which is 8 way set associative, and 256 KB in size. The access latency of the L2 cache is set to 20 cycles. Finally, we connect the L2 cache to 512 MB of DDR5-6400 main memory. With a page size of 256 bytes, the bus width is 8 bits.

2.2 Optical Main Memory

Our first significant change replaced the entire memory subsystem with a large optical main memory, as in Figure 6b. To create an optical memory in gem5, we created a Python class OpticalMainMem,

which extends the DRAMInterface class. Since the DRAMInterface class is built to simulate DRAM devices, there are several components of which we either changed or removed to model our optical devices. Firstly, because optical memories do not leak charge over time like DRAM devices do, we removed the refresh functionality from our OpticalMainMem class. To accomplish this, we modified two refresh timing parameters. We set the tRFC parameter to 0 nanoseconds. If a refresh is initiated, this parameter controls the latency of the refresh. By setting it to 0 nanoseconds, there is no delay when the interface deems it needs to refresh. We also altered the tREFI parameter, to 10,000 seconds. This parameter controls the maximum amount of time a DRAM device can sustain its values before needing to refresh. At such an arbitrarily large value, we essentially guarantee that our memory will never need to self-refresh during the course of our simulations. Secondly, we decreased the latencies of many of the other operating parameters to allow the data to be ready at the row buffer in time to be consumed over the data bus, at our operating frequency of 20 GHz. Lastly, we modify the storage capability and the bandwidth of the memory. Since optical memories allow for higher bandwidths over traditional electronic memories, we opted to set the bus width at 256 bits and a 2 KB row buffer. These values were selected to be specifically wider than the HBM classes provided by gem5, but at a scale we think is still reasonable. We sized this optical main memory at 256 MB, and connected it directly to the processor.

2.3 Optical Caches

Our second significant change replaces only the cache hierarchy in a single core with a pair of optical caches, one for instructions, and one for data, as showcased in Figure 6c. With this model, we chose to leave the main memory at the DDR5-6400 model. To have a similar comparison to the traditional hierarchy, we decided to model the optical caches similar to the L2 cache. Each of the caches are 8 way set associative, and 128 KB in size, totaling to 256 KB together, which equals the size of the original L2 cache. gem5 makes it very straightforward to modify the latency of a cache, so likewise to the optical main memory simulations, we set the hit latency of the optical caches to 50 picoseconds.

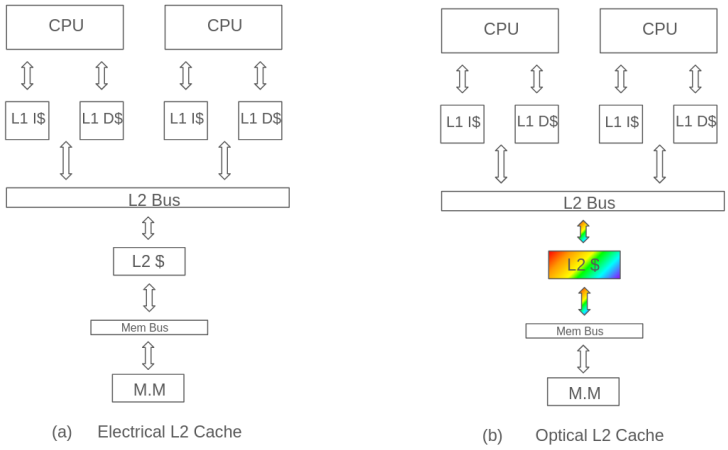

Figure 7: Collection of Multiprocessor Hierarchies

2.4 Optical Caches in Multiprocessors

Our third proposed change is in the multiprocessing domain. Our traditional multiprocessing hierarchy is very similar to the single core hierarchy (Figure 7a), however, with more cores. Each core has a private L1 instruction cache and a private L1 data cache. Each of these private caches is connected to an L2 cache, which is shared among all cores. The associativity, capacity, and latency of these caches are identical to what they are in the single core traditional hierarchy. Once again, the main memory is modeled as the DDR5-6400 model.

In the multiprocessor hierarchy, our aim is to replace the L2 cache with an optical L2 cache, which we have represented in Figure 7b. The hit latency will remain at 50 picoseconds and the cache size will remain as before. The working principle for evaluating optical caches in a multiprocessor set up is that the heavily decreased latency will allow each processor to access the L2 cache in a given 4 GHz clock cycle if necessary, significantly reducing latency and consistency overheads.

3 Tasks and Contributions

3.1 Task 1: Collection of Physical Parameters

As the research group that Parth is a part of works with optical memories, Parth discussed with his peers to collect data from their research regarding the area, power and performance characteristics of the optical memories they work with. Outside of the research that Parth's group performs, he also examined in greater detail literature surrounding optical memories, to gather a sense of what physical technologies currently exist, and what their area, power and performance parameters are. With the larger set of data Parth had gathered, we were now ready to begin modelling our findings in gem5.

3.2 Task 2: Creation of Optical Memory Models in gem5

After Parth had collected the optical memory parameters, Aidan created the optical models in gem5. The optical cache models were very straightforward, since it just required changes to the cache access latencies, but the optical main memory model required more work. Initially, after a few preliminary tests with modifying the DRAMInterface Python class, we thought we needed to directly modify the dram_interface C++ source code. Both Parth and Aidan attempted to understand the functionality of the source code, so that we could modify it to meet our needs, however, after Aidan went back to the Python scripts, he analyzed in greater depth the timing parameters that gem5 has, and better tuned them to the functionality we desired. After this, we noticed our optical main memory model began performing as expected.

3.3 Task 3: Creation of Configuration Scripts

We needed to create Python configuration scripts in gem5, which would allow us to make the changes necessary to run each of our different hierarchies. Initially, Aidan created a configuration script which allowed us to run each of our single processor setups. We also needed to be able to support multiprocessing, so Parth created a new, separate configuration script which instantiated multiple cores, and their private L1 caches. This new script, similar to Aidan's, allowed us to modify the rest of the memory

subsystem to compare our optical memories to traditional memories.

3.4 Task 4: Modifying Hopscotch for gem5 Compatibility

The Hopscotch Benchmark is great, but it needed several modifications to run in gem5. Firstly, since some of the optimizations that Hopscotch enables uses opcodes that gem5 does not support, Aidan firstly began exploring to find out how to add the missing opcodes. Instead, Aidan decided to declare several variables in the benchmark as volatile to prevent such optimizations from occurring. We wanted to support gem5's stats dumping function calls, so Aidan worked to modify the Hopscotch Makefiles to include gem5's m5ops library. After compiling Hopscotch with m5ops, Aidan heavily modified the kernel launch macro, as the macro originally runs a kernel for a specified number of seconds. We modified this functionality to only run a predefined number of iterations, as the syscall emulation for time kernels kept the program from properly completing. Finally, Parth explored how to utilize OpenMP to parallelize the kernels, and to dump stats around the execution of the kernels on a per-core basis. Since gem5 simulation is a long process as it is, we reduced the working set size for the Hopscotch benchmark from 1 GB to 1 MB. We do not think this change in value significantly altered our final results, as 1 MB is still larger than the size of the caches in any of our system configurations.

3.5 Task 5: Collecting and Interpreting Data

Once we had all of the changes implemented in gem5, along with our modified Hopscotch benchmarks, both Parth and Aidan made use of their personal computers to run gem5 simulations. Aidan primarily focused on running single processor simulations, and Parth focused on running multiprocessor simulations. Once we ran each of our simulations, we worked to compile the results into a spreadsheet, where we worked together to interpret the results, and created charts that effectively portrayed our results.

4 Results

Since we evaluated many different system configurations, we will split the results section into several subsections. First, in Section 4.1, we will briefly review our initial tests on an in order processor. Then,

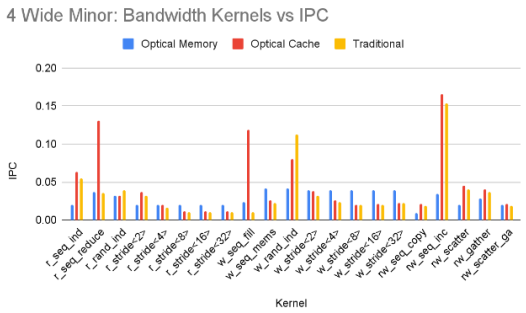

Figure 8: 4-Wide In Order CPU Results for Each Kernel

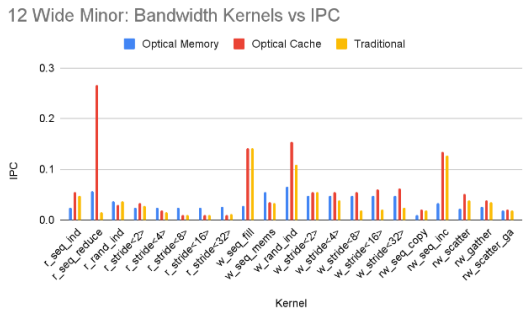

Figure 9: 12-Wide In Order CPU Results for Each Kernel

in Section 4.2, we will analyze the performance of optical memories in an out of order processor. In Section 4.3, we will consider the area cost of our optical memories and provide a case for why we explore optical caches in multiprocessors. Finally, in Section 4.4, we will look at the results of multiprocessor systems.

4.1 In Order Superscalar CPU Results

The results for a 4-wide in order CPU (using a MinorCPU variant in gem5) can be seen in Figure 8. In Figure 9, we showcase the results for the same benchmark programs, but for a 12-wide in order CPU. Besides a few outliers where caching seems to net us a relative boost, performance for both 4-wide and 12-wide in order processors are stuck around 0.05 IPC, irrespective of which memory subsystem is in use. We attribute this lack of performance due to the nature of in order processors. The instruction dependencies and non-memory instruction latencies happen to dominate the execution flow of an in order processor, so our novel memories are not able to prove their worth.

4.2 Out of Order Superscalar CPU Results

When we run the same benchmarks in a 4-wide (Figure 10) and 12-wide (Figure 11) out of order processor using a wider O3CPU variant in gem5, we see more significant results. In particular, we see that for most read kernels, the optical main memory model is 0.4 IPC better in 4-wide processors, and above 1 IPC better in 12-wide processors. Although write performance is still slow with optical main memory, it is across the board faster than other models we simulated. We will later reason as to why write

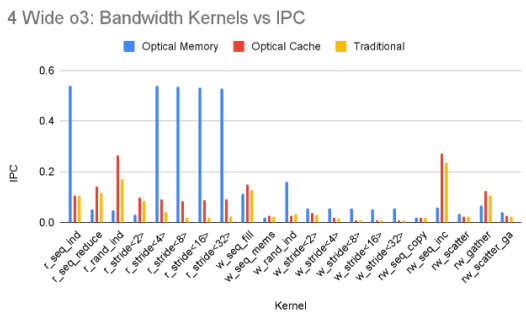

Figure 10: 4-Wide Out of Order CPU Results for Each Kernel

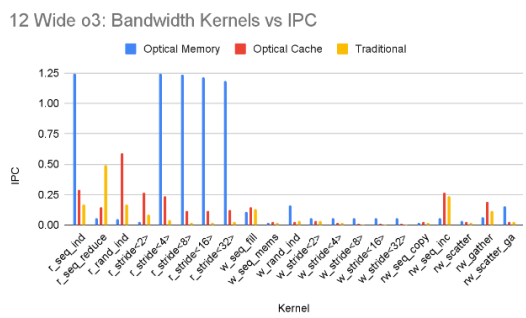

Figure 11: 12-Wide Out of Order CPU Results for Each Kernel

performance may not be as impressive as read performance in the Future Work section.

We also see from these results that the optical caches perform only marginally better than the traditional caches, in almost every benchmark. The reason that optical caches may not perform significantly better than traditional caching in a single core system is for that reason; it is a single core, so there is only one consumer, thus the caches need be only as fast as the processor itself. The performance benefit we do see, however, is the difference in performance where the traditional system misses in the L1 layer, but hits in the L2 layer. Both hierarchies have similar delays when they need to access main memory.

We see considerable performance benefits from a higher processor width with out of order execution and an optical main memory. We attribute this benefit to the increased processor width no longer being as bound on memory instruction latency, so the larger instruction window serves the out of order engine better, as it is able to do more compute, rather than waiting on memory instructions to finish.

4.3 Area Considerations

Although we see significant performance benefits with optical main memories, we find that implementing them in a real system is currently infeasible. With the area parameters gathered in Table 1, 256 MB of DRAM takes up a total of 1.87 mm^2, whereas 256 MB of optical memory takes up 0.246 m^2. Unless we employ die stacking, or some other technology not currently available with optical memories, we cannot use optical memory as main memory until significant advancements are made with the technology.

However, 256 KB of optical memory only requires 0.252 mm^2 of area. While still considerably large,

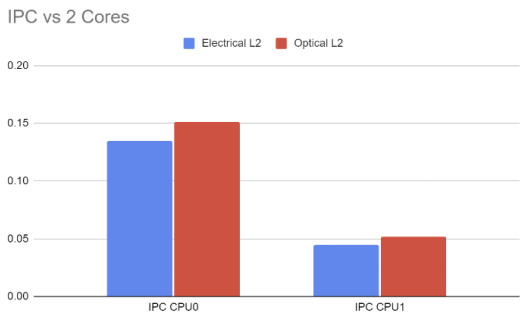

Figure 12: 2-core Results, For Each Core

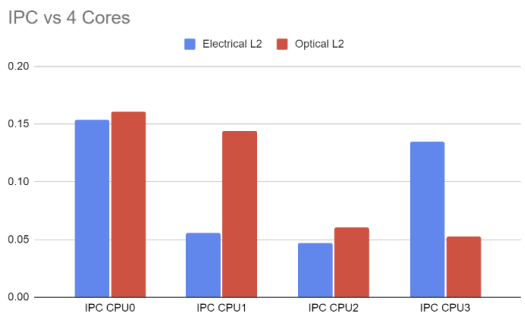

Figure 13: 4-core Results, For Each Core

the size is reasonable enough to fit on a single package. Thus, since 256 KB is the working size of our optical caches, we find that optical caching might be within reason. Since chiplet designs allow for differing node sizes, and a greater total area than monolithic circuit design [23], optical caches might make sense in a multiprocessing system, if placed as a cache chiplet, and if performance is acceptable.

4.4 Multiprocessing Results

With area considerations in mind, and since L1 caches already have single cycle latencies, we replace the L2 cache with an optical cache. The workloads which we gave each processor vary between the 2- and 4-core implementations. In the 2-core system, CPU0 executes the read benchmarks, and CPU1 executes the write and R/W benchmarks. In the 4-core system, CPU0 executes some of the read programs, CPU1 a mix of read and write, CPU2 executes write benchmarks, and finally CPU3 executes R/W programs only.

In Figure 12, we note that there are performance benefits to using optical caches, and similarly in Figure 13, beside CPU3, we also see performance improvements. Most significantly, we see an almost 0.1 IPC boost on CPU1 in the 4-core system. While there are benefits to using an L2 optical cache, we posit that memory consistency serve as the driving factors in multiprocessor memory performance, and replacing the L2 cache does not resolve the consistency problem.

5 Related Work

We did a survey of how novel memory technologies have been integrated with computer architecture simulation tools to demonstrate the performance benefit of those technologies. We looked at SpinSim, a novel computer architecture-level performance evaluation framework for STT-MRAM (Spin-Transfer Torque Magnetic Random Access Memory). SpinSim simulation was done by combining Monte Carlo simulations for bit-cell and peripheral circuit variations with the GEM5+NVMain co-simulator to evaluate system-level performance. The framework simulated a dual-core processor with a 1MB STT-MRAM main memory and benchmarks it using workloads from the PARSEC 2.1 suite [24]. We decided to implement a similar idea by combining data gathered from optical memory research and using it to change parameters in gem5. Several other works [25] use NVMain+Gem5 to simulate power and performance data. Due to time constraints, we decided to stick to only gem5 simulations. Next, we studied the work on optical caches and their performance benefits [26]. They investigated the use of optical memory as an off-chip shared L1 cache in chip multiprocessors (CMPs) connected via optical waveguides. Traditional CMPs employ dedicated L1 caches per core and a shared L2 cache, but the proposed optical architecture uses a single-level shared L1 optical cache. The design leveraged the high speed of optical caches to serve multiple core requests concurrently, eliminating data consistency issues associated with dedicated L1 caches. The shared optical cache simplified coherency management, avoiding L2 cache overhead, and reduced total capacity requirements by 1.84x. The study concluded that optical caches offer substantial performance gains and area benefits, particularly for data-intensive applications, while future work will explore additional benchmarks and cache associativity impacts. We decided to implement a similar idea by modifying latency in gem5. Along with the aforementioned works, we have done an extensive literature survey to find out the current SRAM, DRAM speeds and corresponding various optical memory technology parameters. Several references for those have been given in the background section. We have used the concept of extracting paramaters from literature/simulations for a memory technology and implementing the corresponding changes in an architecture simulation tool to evaluate against application specific benchmarks as seen in several works.

6 Conclusion

We analyzed various types of CPUs, including in-order and out-of-order architectures with different widths, by modifying the memory hierarchy in two ways. In the first scenario, we replaced the entire cache system with optical main memory, while in the second scenario, we replaced only the L1 cache with an optical cache to evaluate performance improvements. While the optical L1 cache showed some improvement in Instructions Per Cycle (IPC), the optical main memory delivered the highest performance gain overall. As illustrated in Figure 14, the most significant benefit was observed for wider out-of-order CPUs, where memory access becomes the primary bottleneck for parallel processing. The increased speed and bandwidth of optical memory effectively addressed this bottleneck. For workloads with a large number of memory accesses, implementing even wider processors could provide significant parallel processing benefits, potentially enabling faster computing as technology advances. However, this improvement comes with an area tradeoff. Given the current state of optical memory technology, integrating such a large optical main memory onto a single chip remains impractical. To strike a balance between performance and area efficiency, we explored an alternative approach. In multiprocessor systems, replacing the L2 electronic cache with an optical cache resulted in noticeable performance gains. In conclusion, faster memory alone does not automatically lead to a better-performing system. It is essential to take a holistic approach and carefully evaluate which part of the memory hierarchy to replace for optimal results. Photonic computing holds the potential to revolutionize computing, and our project highlights its promising benefits.

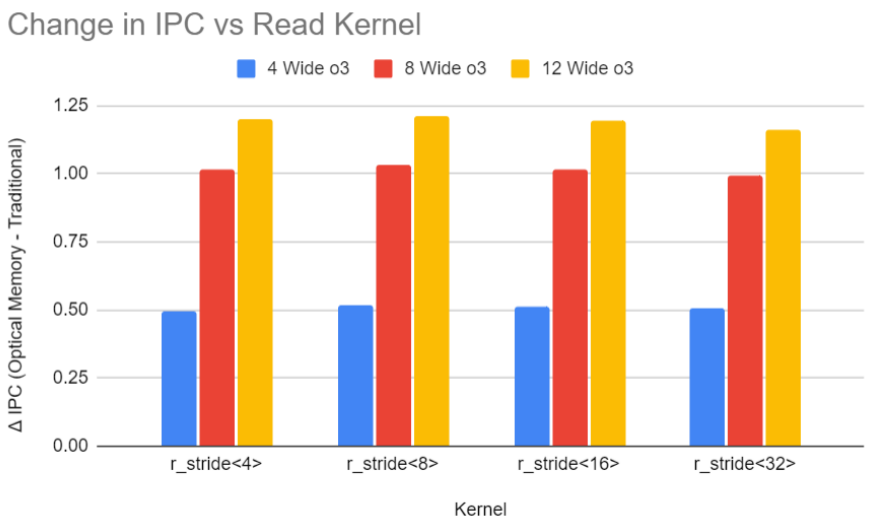

Figure 14: IPC Change with increasing width for OoO Processor

7 Future Work

In this report, we briefly touched upon the power implications of using optical memories as a replacement to the electronic solutions the industry has currently embraced. In the future, we would like to analyze in greater detail the power metrics of optical memory. In particular, we wish to analyze the power characteristics of keeping an optical row buffer open, and what the energy cost is specifically to read or write to the memory. What might the power metrics look like to read a value at the row buffer, when considering how long the bitlines and wordlines must be in an optical memory?

As was seen previously, although the read performance saw significant improvements, the write performance saw minimal improvements. We were pointed in the direction of work that the Lowe-Power group at the University of California-Davis has done, and inconsistencies with the LSQ implementation in gem5. The group has created a gem5 configuration script to model the Intel Skylake architecture with better accuracy, however, we were unable to get the configuration script to work along side our changes in time to create this report. As far as future work in gem5, we would likely start with this script, as well as further analyzing the LSQ, to see if the write problems are due to the LSQ, or native to our optical memory implementation.

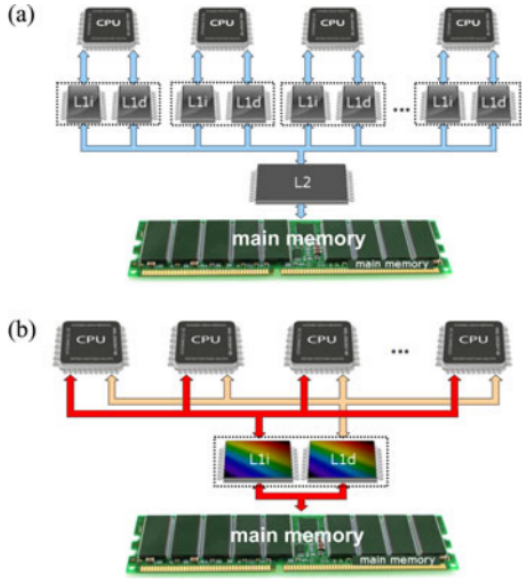

Figure 15: Optical caches in the multiprocessing

Maniotis et al. also explored the possibilities of using optical caches in the multiprocessing stack as seen in Figure 15 [26]. However, they replaced the entire cache hierarchy with a single, unified optical cache, similar to our proposed solution for single core optical caches. To the best of our knowledge, gem5 requires each core to have a set of private L1 caches, but as future work, if we can remove or work around this restriction, we would like to explore the efficacy of a singular, unified optical cache in gem5. There are potential performance implications to this, with coherence being a major component. Similar to the future work suggested by Maniotis, we would also explore the limitations of cache associativity and optical memories.

Photonic interconnects are becoming increasingly popular in high-speed computing [27] with Google's TPUs attempting to integrate them to obtain better performance in ML applications. We theorize that it will be easier to integrate optical memories with photonic interconnects because it will have the potential to delay the conversion of optical signals to electrical signals, thereby increasing speed. The combination of these two technologies could give us exponential benefits.

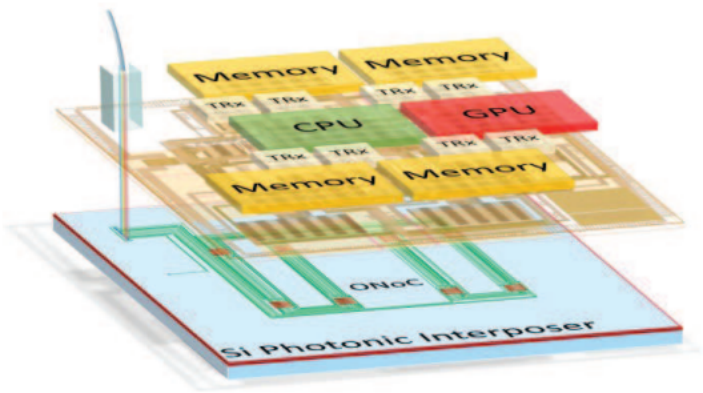

Figure 16: Photonic Interconnects

References

[1] Wm A Wulf and Sally A McKee. Hitting the memory wall: Implications of the obvious. *ACM SIGARCH computer architecture news*, 23(1):20–24, 1995.

[2] Hadi Esmaeilzadeh, Emily Blem, Renee St. Amant, Karthikeyan Sankaralingam, and Doug Burger. Dark silicon and the end of multicore scaling. In *Proceedings of the 38th annual international symposium on Computer architecture*, pages 365–376, 2011.

[3] Scott E Thompson and Srivatsan Parthasarathy. Moore's law: the future of si microelectronics. *Materials today*, 9(6):20–25, 2006.

[4] Thomas N Theis and H-S Philip Wong. The end of moore's law: A new beginning for information technology. *Computing in science & engineering*, 19(2):41–50, 2017.

[5] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *nature*, 521(7553):436–444, 2015.

[6] Amir Gholami, Zhewei Yao, Sehoon Kim, Coleman Hooper, Michael W Mahoney, and Kurt Keutzer. Ai and memory wall. *IEEE Micro*, 2024.

[7] Laszlo Gyongyosi and Sandor Imre. A survey on quantum computing technology. *Computer Science Review*, 31:51–71, 2019.

[8] Themistoklis Prodromakis and Chris Toumazou. A review on memristive devices and applications. In *2010 17th IEEE international conference on electronics, circuits and systems*, pages 934–937. IEEE, 2010.

[9] Steve Furber. Large-scale neuromorphic computing systems. *Journal of neural engineering*, 13(5):051001, 2016.

[10] Xiao-Yun Xu and Xian-Min Jin. Integrated photonic computing beyond the von neumann architecture. *ACS Photonics*, 10(4):1027–1036, 2023.

[11] James D Meindl. Beyond moore's law: The interconnect era. *Computing in Science & Engineering*, 5(1):20–24, 2003.

[12] Keonhee Cho, Heekyung Choi, In Jun Jung, Jisang Oh, Tae Woo Oh, Kiryong Kim, Giseok Kim, Taemin Choi, Changsu Sim, Taejoong Song, et al. Sram write-and performance-assist cells for reducing interconnect resistance effects increased with technology scaling. *IEEE Journal of Solid-State Circuits*, 57(4):1039–1048, 2022.

[13] Lina M Shaker, Ahmed Al-Amiery, Wan Nor Roslam Wan Isahak, and Waleed Khalid Al-Azzawi. Integrated photonics: bridging the gap between optics and electronics for enhancing information processing. *Journal of Optics*, pages 1–13, 2023.

[14] Caxton Griffith Kibebe, Yue Liu, and Jiaxi Tang. Harnessing optical advantages in computing: a review of current and future trends. *Frontiers in Physics*, 12:1379051, 2024.

[15] Peter J Winzer. Modulation and multiplexing in optical communications. In *Conference on Lasers and Electro-Optics*, page CTuL3. Optica Publishing Group, 2009.

[16] Nikos Pleros, Dimitrios Apostolopoulos, Dimitrios Petrantonakis, Christos Stamatiadis, and Hercules Avramopoulos. Optical static ram cell. *IEEE Photonics Technology Letters*, 21(2):73–75, 2008.

[17] Chin-Hui Chen, Shinji Matsuo, Kengo Nozaki, Akihiko Shinya, Tomonari Sato, Yoshihiro Kawaguchi, Hisashi Sumikura, and Masaya Notomi. All-optical memory based on injection-locking bistability in photonic crystal lasers. *Optics Express*, 19(4):3387–3395, 2011.

[18] Theoni Alexoudi, George Theodore Kanellos, and Nikos Pleros. Optical ram and integrated optical memories: a survey. *Light: Science & Applications*, 9(1):91, 2020.

[19] Andrey A Nikitin, Ilya A Ryabcev, Aleksei A Nikitin, Alexandr V Kondrashov, Alexander A Semenov, Dmitry A Konkin, Andrey A Kokolov, Feodor I Sheyerman, Leonid I Babak, and Alexey B Ustinov. Optical bistable soi micro-ring resonators for memory applications. *Optics Communications*, 511:127929, 2022.

[20] Ramesh Kudalippalliyalil, Sujith Chandran, Ajey P Jacob, and Akhilesh Jaiswal. Towards scalable, energy-efficient and ultra-fast optical sram. *arXiv preprint arXiv:2111.13682*, 2021.

[21] Sadra Rahimi Kari, Carlos A Ríos Ocampo, Lei Jiang, Jiawei Meng, Nicola Peserico, Volker J Sorger, Juejun Hu, and Nathan Youngblood. Optical and electrical memories for analog optical computing. *IEEE Journal of Selected Topics in Quantum Electronics*, 29(2: Optical Computing):1–12, 2023.

[22] Alif Ahmed and Kevin Skadron. Hopscotch: a micro-benchmark suite for memory performance evaluation. *The International Symposium on Memory Systems*, pages 167–172, 2019.

[23] Samuel Naffziger, Noah Beck, Thomas Burd, Kevin Lepak, Gabriel H. Loh, Mahesh Subramony, and Sean White. Pioneering chiplet technology and design for the amd epyc™ and ryzen™ processor families : Industrial product. pages 57–70, 2021.

[24] Haoyuan Ma, You Wang, Rashid Ali, Zhengyi Hou, Deming Zhang, Erya Deng, Gefei Wang, and Weisheng Zhao. Spinsim: A computer architecture-level variation aware stt-mram performance evaluation framework. In *2021 IEEE International Symposium on Circuits and Systems (ISCAS)*, pages 1–5. IEEE, 2021.

[25] Manu Komalan, Oh Hyung Rock, Matthias Hartmann, Sushil Sakhare, Christian Tenllado, José Ignacio Gómez, Gouri Sankar Kar, Arnaud Furnemont, Francky Catthoor, Sophiane Senni, et al. Main memory organization trade-offs with dram and stt-mram options based on gem5-nvmain simulation frameworks. In *2018 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 103–108. IEEE, 2018.

[26] P. Maniotis, D. Fitsios, G. T. Kanellos, and N. Pleros. Optical buffering for chip multiprocessors: A 16ghz optical cache memory architecture. *Journal of Lightwave Technology*, 31(24):4175–4191, 2013.

[27] Ashok V Krishnamoorthy, Ron Ho, Xuezhe Zheng, Herb Schwetman, Jon Lexau, Pranay Koka, GuoLiang Li, Ivan Shubin, and John E Cunningham. Computer systems based on silicon photonic interconnects. *Proceedings of the IEEE*, 97(7):1337–1361, 2009.